

Introduction

Palindrome problems show up constantly in coding interviews because they force you to think about two skills at once: string manipulation and optimization strategy. LeetCode 2472, Maximum Number of Non-overlapping Palindrome Substrings, looks at first glance like a job for dynamic programming — "maximize a count of non-overlapping pieces" is the classic setup for a DP table or an interval-scheduling algorithm. That's exactly why it's a good interview question. It tests whether you'll reach for the heaviest tool available or actually think about the structure of the problem first.

The payoff here is a greedy algorithm that runs in roughly $O(n \times k)$ time, needs no extra memory beyond a counter and a pointer, and takes about ten lines to implement once you understand why it works. If you're preparing for interviews, this problem is worth internalizing not just for the LeetCode number, but for the pattern it teaches: when you're picking non-overlapping pieces to maximize a count, the *smallest valid piece* usually beats the *biggest available piece*, because it leaves the most room for what comes after.

Problem Overview

You're given a string `s` and an integer `k`. Your job is to select as many non-overlapping substrings of `s` as possible, subject to two rules:

- Every substring you pick must be a palindrome (it reads the same forwards and backwards).
- Every substring you pick must have a length of at least `k`.

Substrings can't overlap — once you use a set of characters for one palindrome, none of those characters can be reused in another. You want to return the maximum number of substrings you can select under these constraints.

The string length can be up to 2,000 characters, and `s` consists only of lowercase English letters, so there's no need to worry about casing or punctuation when checking for palindromes.

Example

Take `s = "abaccdbbd"` and `k = 3`.

Scanning from the left, "aba" (indices 0–2) is a palindrome of length 3, satisfying the minimum length requirement. That's a valid pick. Moving further along, "dbbd" (indices 5–8) is a palindrome of length 4. These two substrings don't share any characters, so they're both valid picks — and you can't find a third non-overlapping palindrome of length 3 or more anywhere else in the string.

Output: 2

Every character not consumed by "aba" or "dbbd" (the "cc" in the middle) either isn't part of a long-enough palindrome on its own, or would require reusing characters already claimed. Two is the maximum.

Intuition

The first instinct most people have is: find the *longest* palindrome available and take it, since a longer palindrome feels like it's "worth more." That instinct is wrong, and it's wrong in an instructive way.

Consider a string that is entirely one giant palindrome. If you greedily grab the biggest palindrome first, you consume the whole string in one move and end up with a count of 1. But that same string might be decomposable into three or four smaller palindromes, each meeting the minimum length requirement, giving you a count of 3 or 4 instead. Taking the "best" single piece cost you several smaller wins.

The fix is to flip the greedy strategy: scan left to right, and the moment you spot *any* palindrome that's at least length k , take the shortest one available and move past it immediately. Don't hold out for something bigger. This works because a smaller palindrome uses fewer characters, which leaves more of the string free for future picks — and since your only goal is maximizing *count*, not total length, using fewer characters per pick is strictly better or equal.

The natural next question is: which lengths do you actually need to check? A palindrome can be peeled — strip the two outer matching characters off any palindrome and what remains in the middle is still a palindrome, just two characters shorter. If you keep peeling, you eventually land on something of length k or $k + 1$, sitting at nearly the same starting position as the original. That means you never need to test every possible palindrome length starting at each index — checking length k and length $k + 1$ is enough to catch the smallest valid palindrome anchored there, regardless of how long the "full" palindrome at that position might be.

This is also why the usual DP-plus-interval-scheduling instinct is overkill: you don't need every palindromic substring in the string, just the shortest one starting at each position, and a swap argument shows using anything longer never helps.

Brute Force Solution

Idea: For every possible starting index, check every possible ending index to find all palindromic substrings of length at least k . Then treat this as an interval-scheduling problem — sort all valid palindrome intervals and run a classic non-overlapping interval selection (or dynamic programming over positions) to find the maximum count.

Algorithm: 1. For each pair (i, j) with $j - i + 1 \geq k$, check if $s[i:j+1]$ is a palindrome. 2. Collect all such intervals. 3. Run DP or greedy interval scheduling to select the maximum number of non-overlapping intervals.

Advantages: Conceptually straightforward, reuses well-known interval-scheduling machinery, easy to reason about correctness.

Disadvantages: Generating every valid palindrome interval is $O(n^2)$ pairs, and checking each one naively costs $O(n)$ per pair, giving $O(n^3)$ before you even get to the scheduling step. For n up to 2,000, that's on the order of 8 billion operations — far too slow.

Complexity: Time $O(n^3)$ (or $O(n^2)$ with palindrome precomputation via DP/Manacher, still followed by a scheduling pass); Space $O(n^2)$ to store all candidate intervals.

Optimal Solution

The key insight, established above, is that you never need every palindrome — just the shortest valid one starting at each index, which is always length k or $k + 1$.

That collapses the whole problem into a single greedy left-to-right scan:

1. Start a pointer i at 0.
2. At position i , check if $s[i:i+k]$ (length k) is a palindrome. If yes, count it, and jump i forward by k .
3. If not, check if $s[i:i+k+1]$ (length $k + 1$) is a palindrome. If yes, count it, and jump i forward by $k + 1$.
4. If neither works, advance i by 1 and try again.
5. Repeat until i runs past the end of the string.

No backtracking, no revisiting earlier positions, no need to compare candidate intervals against each other — every decision is final the moment it's made.

Walking the example $s = \text{"abacdbbd"} , k = 3$:

Position	i	Check length 3	Check length 4	Action
0		"aba" → palindrome —		take it, jump to i = 3
3		"ccd" → not	"ccdb" → not	advance to i = 4
4		"cdb" → not	"cdbb" → not	advance to i = 5
5		"dbb" → not	"dbbd" → palindrome	take it, jump to i = 9
9		out of bounds	—	stop

Final count: **2**, matching the expected output.

Python Code

```
class Solution:
    def maxPalindromes(self, s: str, k: int) -> int:
        def is_pal(l: int, r: int) -> bool:
            while l < r:
                if s[l] != s[r]:
                    return False
                l += 1
                r -= 1
            return True

        n = len(s)
        count = 0
        i = 0

        while i <= n - k:
            if is_pal(i, i + k - 1):
                count += 1
                i += k
            elif i + k < n and is_pal(i, i + k):
                count += 1
                i += k + 1
            else:
                i += 1

        return count
```

Code Walkthrough

- `is_pal(l, r)` is a standard two-pointer palindrome check: it walks inward from both ends of the slice and bails out the moment characters don't match. It's nested inside `maxPalindromes` purely so it can read `s` directly without passing it as an extra argument.
- `count` tracks how many valid palindromes we've locked in; `i` is the scan pointer.

- The `while i <= n - k` guard stops the scan once there aren't enough characters left to form a length- `k` palindrome.
- First branch: try the length- `k` window starting at `i`. If it's a palindrome, take it and jump `i` forward by exactly `k` — past every character just consumed.
- Second branch: if the length- `k` window fails, try length `k + 1` (guarded by `i + k < n` so the slice doesn't run off the end of the string). If that's a palindrome, take it and jump forward by `k + 1`.
- Fallback: if neither window is a palindrome, slide `i` forward by 1 and try again from the next starting position.
- The loop never moves `i` backward, which is what keeps this a single linear pass instead of something requiring backtracking.

Dry Run

Using `s = "abaccdbbd"`, `k = 3`, `n = 9`:

- `i = 0`: `is_pal(0, 2)` checks `s[0]='a'` vs `s[2]='a'` (match), pointers cross → palindrome. `count = 1`, `i = 3`.
- `i = 3`: `is_pal(3, 5)` checks `s[3]='c'` vs `s[5]='d'` → mismatch, not a palindrome. Try length 4: `is_pal(3, 6)` checks `s[3]='c'` vs `s[6]='b'` → mismatch. `i = 4`.
- `i = 4`: `is_pal(4, 6)` checks `s[4]='c'` vs `s[6]='b'` → mismatch. Length 4: `is_pal(4, 7)` checks `s[4]='c'` vs `s[7]='b'` → mismatch. `i = 5`.
- `i = 5`: `is_pal(5, 7)` checks `s[5]='d'` vs `s[7]='b'` → mismatch. Length 4: `is_pal(5, 8)` checks `s[5]='d'` vs `s[8]='d'` (match), then `s[6]='b'` vs `s[7]='b'` (match), pointers cross → palindrome. `count = 2`, `i = 9`.
- `i = 9`: loop condition `9 <= 9 - 3` (i.e. `9 <= 6`) is false. Loop ends.

Return `count = 2`. Matches the expected output.

Complexity Analysis

Time: Each palindrome check (`is_pal`) costs up to $O(k)$ steps since it walks inward from both ends of a window of size `k` or `k + 1`. The pointer `i` advances through the string, and in the worst case it moves one step at a time doing an $O(k)$ check at every position, giving $O(n \times k)$ overall. Since `k` is bounded by `n`, this is comfortably fast for `n` up to 2,000.

Space: $O(1)$ beyond the input string — just a counter and a pointer, no auxiliary arrays or recursion stack.

Why it's correct: The peeling argument guarantees the shortest palindrome at any starting index is length k or $k + 1$, so checking only those two lengths never misses a valid pick. The greedy-exchange argument guarantees that taking the shortest available palindrome at the earliest position is never worse than any alternative choice, because any solution using a longer or later palindrome can be modified to use the shorter, earlier one without reducing the total count — it only frees up more string for subsequent picks.

Alternative Solutions

A dynamic programming formulation is possible: define $dp[i]$ as the maximum count achievable using $s[i:]$, and transition by checking, at each i , whether a length- k or length- $k + 1$ palindrome starts there, taking $1 + dp[i + \text{length}]$ versus $dp[i + 1]$. This gives the same $O(n \times k)$ time complexity but requires $O(n)$ space for the DP array and is more code to write and reason about. Since the greedy approach provably makes the optimal local choice at every step (no situation ever benefits from skipping a valid earliest palindrome), the DP array is unnecessary — the greedy scan is a strict simplification with identical complexity, which is why it's the preferred solution here.

Edge Cases

- **Empty or too-short string:** If $\text{len}(s) < k$, the loop condition $i \leq n - k$ is false immediately, and the function correctly returns 0.
- **No palindromes at all meeting length k :** The pointer slides all the way through via the fallback branch, and `count` stays 0.
- **$k = 1$:** Every single character is trivially a palindrome of length 1, so the algorithm greedily takes one-character palindromes back-to-back, maximizing count by consuming the string one character at a time.
- **Entire string is one palindrome:** The algorithm won't fall for the "grab the biggest chunk" trap — it finds the shortest palindrome at the earliest position first, which is exactly the behavior that prevents undercounting.
- **String with repeated identical characters (e.g., "aaaaaa", $k = 2$):** Nearly every adjacent pair is a palindrome, so the greedy scan takes non-overlapping pairs consecutively, maximizing the count for that structure.

Common Mistakes

1. **Always picking the longest available palindrome.** This is the single biggest trap in this problem — it feels intuitive but actively reduces the final count, as shown in the intuition section.

2. **Checking every possible substring length at each position.** This isn't wrong, just wasteful — it turns an $O(n \times k)$ solution into something far slower without improving correctness.
3. **Forgetting the $i + k < n$ bounds check before testing the length- $k + 1$ window.** Skipping this causes an index-out-of-range error near the end of the string.
4. **Advancing the pointer by 1 after a successful match instead of jumping past the full matched length.** This allows overlapping substrings to be selected, violating the problem's core constraint.
5. **Assuming a DP table over all palindromic substrings is required.** Building a full palindrome DP table (`is_palindrome[i][j]` for all pairs) is correct but unnecessary — it costs $O(n^2)$ space and time for information the greedy scan never needs.
6. **Off-by-one errors in the two-pointer palindrome check**, particularly around whether the slice bounds are inclusive or exclusive, which silently produces wrong palindrome verdicts near string edges.

Interview Questions

- Why is it always safe to prefer the shortest valid palindrome over a longer one at the same starting position?
- Can you prove the greedy choice is optimal using an exchange argument?
- How would the solution change if substrings were allowed to overlap?
- What's the time complexity if k is a large fraction of n , and can you improve it using an algorithm like Manacher's for palindrome detection?
- How would you modify the algorithm to also return the actual substrings selected, not just the count?
- What happens if the string can contain uppercase letters or punctuation — how does that change the palindrome check?

Similar Problems

- **LeetCode 5 (Longest Palindromic Substring)** — foundational palindrome-detection practice, though it optimizes for length rather than count.
- **LeetCode 132 (Palindrome Partitioning II)** — a DP problem about minimizing cuts to partition a string into palindromes, a useful contrast to this greedy, count-maximizing variant.
- **LeetCode 435 (Non-overlapping Intervals)** — the general interval-scheduling pattern this problem's greedy strategy echoes.

- **LeetCode 267 (Palindrome Permutation II)** — different angle on palindrome construction, good for reinforcing palindrome properties.
- **LeetCode 647 (Palindromic Substrings)** — counts all palindromic substrings in a string, useful for practicing the two-pointer expand-around-center technique used here.

Key Takeaways

Maximizing a count of non-overlapping pieces almost always rewards greedily taking the smallest valid piece as early as possible, not the largest available one — freeing up space for future picks beats "winning big" on any single pick. For palindrome-specific problems, the peeling property (a palindrome's inner core is itself a shorter palindrome) is a powerful tool for shrinking the search space from "check every length" down to "check the two shortest possible lengths." Recognizing when a DP-and-interval-scheduling instinct is overkill — and replacing it with a provably correct greedy pass — is a skill worth practicing beyond this one problem.

Watch the Video

For the full walkthrough with the visual trace through the example string, the greedy loop built step by step, and two pause-and-guess quizzes to test your understanding before the recap, check out the video here: {LINK}

About the Series

This breakdown is part of Fun with Learning Technology, a series that works through LeetCode problems the way an experienced engineer actually approaches them — starting from intuition and problem structure before ever writing code, and explaining not just the solution but why the "obvious" approach often isn't the right one.

Call To Action

If this walkthrough helped the greedy trick click, give the video a like on YouTube and subscribe for more problem breakdowns like this one. For deeper write-ups and behind-the-scenes notes on how these solutions come together, subscribe to the Substack. Drop a comment with your own approach to LeetCode 2472, or share this with someone prepping for interviews.

Quick Review

You need to pick k non-overlapping palindromic substrings maximizing count — what's the greedy tell?

Interval scheduling greedy: always take the earliest-ending valid palindrome, then jump past it.

Time complexity of the DP + smallest-palindrome-check solution for LC 2472?

$O(n^2)$ — DP over end index times checking substrings of length k and $k+1$ at each position.

Space complexity of the standard DP approach here?

$O(n)$ for the dp array; palindrome checks done in $O(1)$ extra via two-pointer, no full table stored.

Common bug: why not just take the shortest palindrome you find anywhere?

You must take the shortest palindrome ending earliest that meets length k , not any short one — order by end index, not just length.

Why does the algorithm only test lengths k and $k+1$, not longer ones?

Any longer valid palindrome contains a shorter valid one inside it ending no later, so checking k and $k+1$ always dominates.

Interval scheduling (activity selection) vs this palindrome-selection problem — key difference?

Classic scheduling picks earliest-finishing among given intervals; here you must first discover minimal-length palindromic intervals, then apply the same greedy.